

박수민_0615_TIMER FND BTN CONTROL

```
#fnd.c
/*
 * fnd.c
 *
 * Created: 2026-06-12 오후 1:22:04
 * Author: kccistc
 */

#include "fnd.h"
#include "button.h"
#include <avr/interrupt.h>

/* — 모드 정의 — */
#define MODE_CLOCK    0    // MM.SS 시계 (도트 1초마다 깜빡임)
#define MODE_COUNT    1    // 초시계      (오른쪽 초 카운트 + 왼쪽 원형 애니메이션)
#define MODE_SW       2    // 스톱워치   (버튼1로 start/stop)

#define ANIM_PERIOD   100 // 원형 애니메이션 한 칸 이동 주기(ms)

volatile uint32_t ms_count    = 0;    // 시계용 ms
volatile uint32_t sec_count   = 0;    // 시계용 sec
volatile uint8_t  dot_display = 0;    // 시계 가운데 도트
volatile uint32_t cnt_ms      = 0;    // 초시계용 ms
volatile uint32_t sw_ms       = 0;    // 스톱워치 누적 ms
volatile uint8_t  anim_step    = 0;    // 초시계 원형 애니메이션 위치 (0~7)

volatile uint32_t msec_count = 0;    // Timer2 1ms 카운터 (최적화 방지)

extern int get_button(int button_num, int button_pin);

int fnd_main();
void init_fnd();
void init_timer2();
void fnd_display();
void fnd_display_clock(uint32_t total_sec, uint8_t dot_on);
void fnd_display_stopwatch(uint32_t total_ms);
void fnd_display_count(uint32_t total_sec, uint8_t step);
void fnd_all_off();

static const uint8_t fnd_font[] = {
    0xC0, 0xF9, 0xA4, 0xB0, 0x99,    /* 0  1  2  3  4 */

```

```

0x92, 0x82, 0xD8, 0x80, 0x98, /* 5 6 7 8 9 */
0x40, 0x79, 0x24, 0x30, 0x19, /* 0. 1. 2. 3. 4. */
0x12, 0x02, 0x58, 0x00, 0x18 /* 5. 6. 7. 8. 9. */
};

ISR(TIMER2_OVF_vect)
{
    TCNT2 = 194;          // 62tick 뒤 다시 overflow 되도록 reload
    msec_count++;
}

void init_timer2(void)
{
    TCNT2 = 194;          // 1ms 기준 reload 값
    TCCR2 &= ~(1 << CS22) | (1 << CS21) | (1 << CS20); // 분주 비트 reset
    TCCR2 |= (1 << CS22); // 256분주 (ATmega128:
CS22:20 = 100)
    TIMSK |= (1 << TOIE2); // Timer2 overflow 인터럽트 허
용
    sei();                // 전역 인터럽트 허용
}

int fnd_main(void)
{
    uint8_t mode = MODE_CLOCK;
    uint8_t sw_run = 0;    // 스톱워치 동작 여부
    uint32_t last_ms = 0;  // 마지막으로 처리한 msec_count

    ms_count = 0; sec_count = 0; dot_display = 0;
    cnt_ms = 0; sw_ms = 0; anim_step = 0;

    while (1)
    {
        // 1. 화면 갱신 : 매 루프마다 한 자리씩 멀티플렉싱 */
        switch (mode)
        {
            case MODE_CLOCK: fnd_display_clock(sec_count, dot_display);
break;

            case MODE_COUNT: fnd_display_count(cnt_ms / 1000,
anim_step); break;

            case MODE_SW: fnd_display_stopwatch(sw_ms);
break;

        }
    }
}

```

```

/* 2. 1ms tick 이 지났을 때만 시간 진행 + 버튼 처리 */
if (msec_count != last_ms)
{
    last_ms = msec_count;

    ms_count++;
    if (ms_count >= 1000) { ms_count = 0; sec_count++; }
    if (ms_count == 500) { dot_display = !dot_display; }

    if (mode == MODE_COUNT)
    {
        cnt_ms++;
        if (cnt_ms % ANIM_PERIOD == 0)
            anim_step = (anim_step + 1) % 8;
    }

    if (mode == MODE_SW && sw_run) sw_ms++;

    /* 버튼0 : 모드 전환 */
    if (get_button(BUTTON0, BUTTON0PIN))
    {
        switch (mode)
        {
            case MODE_CLOCK: mode = MODE_COUNT;
cnt_ms = 0; break;
            case MODE_COUNT: mode = MODE_SW;
sw_ms = 0; sw_run = 0; break;
            case MODE_SW: mode = MODE_CLOCK;
break;
        }
    }

    /* 버튼1 : 스톱워치 모드에서만 start/stop 토글 */
    if (get_button(BUTTON1, BUTTON1PIN))
    {
        if (mode == MODE_SW)
            sw_run = !sw_run;
    }

    /* 버튼2 : 리셋 등 */
    if (get_button(BUTTON2, BUTTON2PIN))
    {

```

```

        if (mode == MODE_SW)
        {
            if (sw_run == 1)          { sw_ms = 0; sw_run
= 0; } // 달리는 중 -> 리셋+정지

            else if (sw_ms == 0)      { sw_run = 1; }

            // 리셋 상태 -> 시작

            else                        { sw_ms = 0; sw_run
= 0; } // 정지 상태 -> 리셋+정지
        }
        else                          // 다른 모드 -> 전체 리셋
        {
            ms_count = 0; sec_count = 0; dot_display = 0;
            cnt_ms = 0;  anim_step = 0;
            sw_ms = 0;   sw_run = 0;
        }
    }
}

return 0;
}

```

```

void init_fnd(void)
{
    FND_DATA_DDR = 0xff; // 출력모드로 설정한다.
    FND_DIGIT_DDR |= 1 << FND_DIGIT_D1 | 1 << FND_DIGIT_D2 | 1 <<
FND_DIGIT_D3 | 1 << FND_DIGIT_D4;

    // FND를 전체 off 하는 작업
    #if 1          // common anode 방식

        FND_DATA_PORT = 0xff;

    #else          // common cathode 방식

        FND_DATA_PORT ^= 0xff;

    #endif

    FND_DIGIT_PORT
~((1<<FND_DIGIT_D1)|(1<<FND_DIGIT_D2)|(1<<FND_DIGIT_D3)|(1<<FND_DIGIT_D4));
&=

```

```
}
```

```
void fnd_all_off(void)
{
    FND_DIGIT_PORT &= ~((1 << FND_DIGIT_D1) | (1 << FND_DIGIT_D2) | (1 <<
FND_DIGIT_D3) | (1 << FND_DIGIT_D4));
    FND_DATA_PORT = 0xFF;
}
```

```
void fnd_display(void)
{
    static int digit_select = 0;                                // 자릿수 선택
    switch(digit_select)
    {
        case(0):
            // 1단위
            #if 1
                FND_DIGIT_PORT = 0x80;                            / /
anode
            #else
                FND_DIGIT_PORT = ~0x80;
            // cathode
            #endif
            FND_DATA_PORT = fnd_font[sec_count%10]; // 0~9
            break;

        case(1):
            // 10단위
            #if 1
                FND_DIGIT_PORT = 0x40;
            #else
                FND_DIGIT_PORT = ~0x40;
            #endif
            FND_DATA_PORT = fnd_font[sec_count/10%6]; // 0~59
            break;

        case(2):
            // 100단위
```

```

        #if 1
            FND_DIGIT_PORT = 0x20;
        #else
            FND_DIGIT_PORT = ~0x20;
        #endif
        FND_DATA_PORT = fnd_font[sec_count/60%10];
        break;

        case(3):
// 1000단위
        #if 1
            FND_DIGIT_PORT = 0x10;
        #else
            FND_DIGIT_PORT = ~0x10;
        #endif
        FND_DATA_PORT = fnd_font[sec_count/600%6]; // 0~59
        break;
    }
    digit_select = (digit_select + 1) % 4; // 다음 표시할 자리수
}

void fnd_display_count(uint32_t total_sec, uint8_t step)
{
    static uint8_t d = 0;

    uint8_t sec1 = total_sec % 10;
    uint8_t sec10 = (total_sec / 10) % 6;

    /* 각 단계가 어느 칸(자리수), 어느 세그먼트인지 */
    static const uint8_t anim_digit[8] = { 0x20, 0x10, 0x10, 0x10, 0x10, 0x20, 0x20,
0x20 };
    static const uint8_t anim_seg[8] = { 0xFE, 0xFE, 0xDF, 0xEF, 0xF7, 0xF7,
0xFB, 0xFD };

    switch (d)
    {
        case 0: // 초1
            FND_DIGIT_PORT = 0x80; FND_DATA_PORT = fnd_font[sec1]; break;
        case 1: // 초10
            FND_DIGIT_PORT = 0x40; FND_DATA_PORT = fnd_font[sec10]; break;
        case 2: // circle anim
            FND_DIGIT_PORT = 0x20;

```

```

        FND_DATA_PORT = (anim_digit[step] == 0x20) ? anim_seg[step] : 0xFF;
        break;
    case 3: // circle anim
        FND_DIGIT_PORT = 0x10;
        FND_DATA_PORT = (anim_digit[step] == 0x10) ? anim_seg[step] : 0xFF;
        break;
    }
    d = (d + 1) % 4;
}

```

```

void fnd_display_clock(uint32_t total_sec, uint8_t dot_on)
{

```

```

    static uint8_t d = 0;

```

```

    uint8_t sec1 = total_sec % 10;

```

```

    uint8_t sec10 = (total_sec / 10) % 6;

```

```

    uint8_t min1 = (total_sec / 60) % 10;

```

```

    uint8_t min10 = (total_sec / 600) % 6;

```

```

    switch (d)
    {

```

```

        case 0: // 초 1의 자리 (오른쪽 끝)

```

```

        FND_DIGIT_PORT = 0x80;

```

```

        FND_DATA_PORT = fnd_font[sec1];

```

```

        break;

```

```

        case 1: // 초 10의 자리

```

```

        FND_DIGIT_PORT = 0x40;

```

```

        FND_DATA_PORT = fnd_font[sec10];

```

```

        break;

```

```

        case 2: // 분 1의 자리 (도트 위치)

```

```

        FND_DIGIT_PORT = 0x20;

```

```

        FND_DATA_PORT = fnd_font[dot_on ? min1 + 10 : min1];

```

```

        break;

```

```

        case 3: // 분 10의 자리 (왼쪽 끝)

```

```

        FND_DIGIT_PORT = 0x10;

```

```

        FND_DATA_PORT = fnd_font[min10];

```

```

        break;
    }

```

```

    d = (d + 1) % 4;
}

```

```

/* SS.CC 표시 (CC = 1/100초 = 10ms 단위). 자릿수 순서 동일: */

```

```

void fnd_display_stopwatch(uint32_t total_ms)

```

```

{

static uint8_t d = 0;

uint32_t cs    = total_ms / 10;      // 10ms(센티초) 단위로 변환
uint8_t  cs1   = cs % 10;           // 10ms 자리
uint8_t  cs10  = (cs / 10) % 10;    // 100ms 자리
uint8_t  sec1  = (cs / 100) % 10;   // 초 1의 자리
uint8_t  sec10 = (cs / 1000) % 10;  // 초 10의 자리

switch (d)
{
    case 0:                                // 10ms 자리 (오른쪽 끝)
        FND_DIGIT_PORT = 0x80;
        FND_DATA_PORT  = fnd_font[cs1];
        break;

    case 1:                                // 100ms
        FND_DIGIT_PORT = 0x40;
        FND_DATA_PORT  = fnd_font[cs10];
        break;

    case 2:                                // 1초대
        FND_DIGIT_PORT = 0x20;
        FND_DATA_PORT  = fnd_font[sec1 + 10];
        break;

    case 3:                                // 10초대
        FND_DIGIT_PORT = 0x10;
        FND_DATA_PORT  = fnd_font[sec10];
        break;
}
d = (d + 1) % 4;
}

```



```

#button.c
/*
 * button.c
 *
 * Created: 2026-06-12 오후 1:22:26
 * Author: kccistc
 */

#include "button.h"

extern volatile uint32_t msec_count;

void init_button(void);
int get_button(int button_num, int button_pin);

#define DEBOUNCE_MS 20

// 버튼 초기화 (입력 방향)
void init_button(void)
{
    BUTTON_DDR &= ~(1 << BUTTON0PIN | 1 << BUTTON1PIN | 1 << BUTTON2PIN |
1 << BUTTON3PIN);
}

int get_button(int button_num, int button_pin)
{
    // stable : 디바운스로 확정된 상태
    // last_raw : 직전에 읽은 raw 값
    // change_t : raw 값이 마지막으로 바뀐 시각(ms)
    static unsigned char stable[BUTTON_NUMBER] = { BUTTON_RELEASE,
BUTTON_RELEASE, BUTTON_RELEASE, BUTTON_RELEASE };
    static unsigned char last_raw[BUTTON_NUMBER] = { BUTTON_RELEASE,
BUTTON_RELEASE, BUTTON_RELEASE, BUTTON_RELEASE };
    static uint32_t change_t[BUTTON_NUMBER] = { 0, 0, 0, 0 };

    // 1) 현재 핀 raw 값
    unsigned char raw = (BUTTON_PIN & (1 << button_pin)) ? BUTTON_PRESS :
BUTTON_RELEASE;

    // 2) raw 가 바뀌면 그 시각을 기록 (여기서 기다리지 않음)
    if (raw != last_raw[button_num])
    {
        last_raw[button_num] = raw;
    }
}

```

```

        change_t[button_num] = msec_count;
    }

    // 3) raw 가 DEBOUNCE_MS 이상 그대로 유지되면 확정 상태에 반영
    if ((msec_count - change_t[button_num]) >= DEBOUNCE_MS && raw !=
stable[button_num])
    {
        stable[button_num] = raw;

        // PRESS -> RELEASE 로 확정되는 순간 = "한 번 눌렀다 떼"
        if (stable[button_num] == BUTTON_RELEASE)
            return 1;
    }

    return 0;
}

```

```

#main.c
/*
 * 03.TIMER_FND_BTN_CONTROL.c
 *
 * Created: 2026-06-15 오후 3:44:54
 * Author : kccistc
 */

/*
 * main.c
 *
 * Timer2 (8bit) Overflow Interrupt 를 이용해 FND 구동
 * 분주비: 256분주
 */

#define F_CPU 16000000UL
#include <avr/io.h>
// #include <avr/interrupt.h> // sei, ISR 등

#include "button.h"
extern void init_button(void);
extern int get_button(int button_num, int button_pin);

```

```
extern int fnd_main();
extern int init_fnd();
extern void init_timer2(void);

#define MODE_CLOCK 0 /* 분:초 시계 */
#define MODE_CHRONO 1 /* 초시계 */
#define MODE_SW 2 /* 스탑워치 */

int main(void)
{
    init_fnd();
    init_button();
    init_timer2();

    while (1)
    {
        fnd_main();
    }
}
```